Task 4: AI Analytics for CRM — Natural Language to SQL

What this task is really testing

You are building an AI assistant that sits on top of a CRM database. A user types a question in plain English — “what is the total revenue from my clients this quarter?” — and the system generates SQL, runs it, and returns the answer in whatever format best fits the result.

The interesting part is not making it work. With modern AI tooling, a rough natural-language-to-SQL loop is achievable in under an hour. The interesting part is making it **safe**: a Team Member must never be able to see another Team Member's contacts, no matter how they phrase the question, and no matter what the language model decides to generate.

That is the core of this exercise. We are not going to evaluate whether you can code a CRM, that is by default -- AI can do that better than humans. We are evaluating your security architecture, your judgment about where to place trust boundaries, and how effectively you use AI tooling to move fast on everything that isn't a judgment call.

Scope: what is mandatory vs. optional

Mandatory core — build these

1. The CRM data model (contacts, tasks, invoices, estimates, activities, attachments, users) with seed data covering at least 3 users across the 3 roles, 10+ contacts, and enough invoices/tasks to make aggregate questions meaningful.
2. Real authentication (login, hashed passwords, session or JWT) with three roles: Admin, Manager, Team Member.
3. Role-scoped access to contacts and everything hanging off them, **enforced server-side**.
4. The AI assistant: natural language question in, generated SQL, executed query, result rendered back in a sensible format.
5. **Security safeguards on the SQL path.** This is non-negotiable and is weighted most heavily. See the PRD's security section.
6. A logging table capturing every AI query: who asked, the question, the generated SQL, whether it was blocked, tokens consumed, and estimated cost.

Optional — only if core is solid

- Admin panel: user management (create/deactivate/change role)
- Admin panel: token and cost usage analytics with per-user breakdown
- Export to CSV / Markdown
- Search and filter on the contact list UI
- Multiple output formats (table / list / scalar / markdown) chosen contextually
- Per-user or per-role spend limits with soft warning and hard cutoff

A submission with a bulletproof security layer with visible CRM and AI Assistant and no admin panel scores considerably higher than one with a polished admin panel and AI agent that does not work

Tech stack & setup

- Frontend: Next.js
- Backend: Node or Python or Laravel, your choice

- Database: PostgreSQL or MySQL — PostgreSQL is recommended, for reasons the PRD explains
- Any LLM provider you like (Claude, OpenAI, a local model). Use your own API key; Once you push the code, we will use our API key to evaluate. You just have to mention in the env file which model's API key is needed.
- The full stack must come up with **Docker and Docker Compose**. A single command should get a reviewer to a working, seeded application.
- Include a README with login credentials for each of the three seeded roles so a reviewer can test the access boundaries immediately.

Constraints & expectations

- You may split it across sittings; record all of it.
- You are expected to use AI tools (Claude, ChatGPT, Copilot, Cursor, etc.) exactly as you would on a real project — including letting them write your schema, Docker config, migrations, and UI. We are not testing whether you can work without AI.
- You will not finish everything. That is by design. We want to see what you choose to protect when the clock is against you.
- If you disagree with something in the PRD, say so and do it your way — explain your reasoning on the recording. A well-argued deviation is a positive signal, not a negative one.

Recording instructions

Record your screen and your voice for the full session. As you work, narrate:

- Your scoping decisions, before you act on them — what you're building, what you're deliberately skipping, and why
- What you're asking the AI for. Read prompts aloud where it helps, particularly when you delegate a whole chunk of work
- What you accept, reject, or rewrite from AI output — above all in the security-critical code, where a plausible-looking mistake is expensive
- **Your own attempts to break your access control.** Log in as a Team Member and try to reach another user's data through the AI assistant. Try a prompt injection. Show us what happens.
- Anything the AI got wrong that you caught, and how you caught it

Dead ends, changing your mind, and thinking out loud are all fine and expected. We would much rather watch how you actually work than a rehearsed performance.

Submission

- Recording link (Loom / Drive / unlisted YouTube) and code link (GitHub repo or zip)
- docker-compose.yml plus any .env.example needed to run it — we should not have to guess at configuration
- README with the three sets of seeded login credentials
- A short note (a few paragraphs is plenty) covering: what you built, what you skipped and why, the security model you chose, and what you would do next with another four hours
- Send everything to the HR contact who shared this task with you